

PRECOG LAB
NLP TASK
The Ghost in the Machine
–Submitted by Srinidhi Ranjini Girish

Approach and Current Status of the task:

This experiment is mainly attributed towards developing and experimenting various models for machine generated text detection. It goes one step further and tries to identify and classify texts that closely mimic human writing styles. The structure of the experiment is closely aligned with the task document with certain design decisions and changes based on literature. The experiment is completed till genetic algorithm with analysis on imposter paragraphs. The below report is written in the format of methodology, results and inference for each stage.

Summary of results

Metric	Class 1	Class 2	Class 3
Type Token Ratio	0.2234	0.1281	0.1412
Hapax Legomena	981	1159	1105
POS Distribution	0.420	0.499	0.564
Dependency Tree Depth	4.7011	7.7351	7.006
Readability Indices	7.9855	15.7348	15.1044

Model	Accuracy
Random Classifier	85.5 %
FeedForward Network	95%
Roberta finetuned with LoRA	98.5 %

Research Report

1. Data collection

The task involves classification of text into three classes

1. Human written texts
2. AI text closely mimicking human styles
3. General AI text

The human written task was taken from novels of Charlotte Bronte downloaded from project gutenber. The choice of the author was purely personal because of the familiarity of the text, thereby helping in streamlining the topics and understanding the style of writing.

1.1 Human written texts

Novels downloaded from Project Gutenberg had a lot of boilerplate code and other details like prologue and chapter heading that weren't required so the first step was to clean the document by performing a set of sequential steps.

Data Cleaning

- > Gutenberg Boilerplate removed
- > Starts from chapter no prelogue or forward
- > Removed tab space
- > Segregated by paragraph
- > removed transcriber notes / footnotes

This ensured that the document only consisted of the author's voice

1.2 AI text Generation

1.2.1 Identification of themes written by charlotte bronte

Themes identified:

1. Love, Marriage & Romantic Relationships
2. Independence, Self-Determination & Ambition
3. Social Class & Class Conflict
4. Gender Roles & Constrained Opportunity
5. Religion & Morality
6. Isolation, Displacement & Belonging
7. Grief, Loss & Endurance

Theme Identification methodology:

The prominent themes in each book were identified using book summaries, literature reviews published across different timelines considering themes from both the 19th century and the themes that evolved later as part of societal growth. The below table has the themes identified in each book and the resources used to identify them.

Jane eyre	Villette	Professor	Shirley
Love, Family, and Independence	Isolation	Self-determination and ambition	Industrialization and social conflict
Social Class and Social Rules	Displacement	Class and materialism vs. intellect	Women's independence and constrained agency
Gender Roles	Gender roles	Labor and industrial conditions	Female friendship
Religion	Cross-cultural conflict	Gender and constrained opportunity	Love and marriage
Feeling vs. Judgment	Religion	Vocation and earned identity	Class and sympathy across divides
The Spiritual and the Supernatural	Resilience	Love complicated by status and rivalry	Religion and moral order
Home and Belonging	Endurance		Grief, loss, and endurance
Anxiety and Uncertainty			

References:

- [https://en.wikipedia.org/wiki/Villette_\(novel\)#Themes](https://en.wikipedia.org/wiki/Villette_(novel)#Themes)
- <https://www.enotes.com/topics/shirley-charlotte-bronte/criticism/criticism/asa-briggs-essay-date-1958>
- <https://www.sparknotes.com/lit/janeeyre/themes/>
- <https://www.litcharts.com/lit/jane-eyre/themes>
- <https://literariness.org/2025/05/13/analysis-of-charlotte-brontes-the-professor/>
- <https://www.ebsco.com/research-starters/literature-and-writing/professor-analysis-major-characters>

1.2.2 Topic Identification

The themes give broader aspects of the text. But to generate AI texts similar to the human text, we need topics that align with the topics discussed in the paragraphs of the book. The themes and the excerpts from the book were used to generate these topics using Gemini LLM

Jane Eyre

1. The Cost of Moral Integrity
2. The Search for Belonging
3. The Ethics of Dependency
4. Class and the Boundaries of Love
5. Faith and Desire
6. The Authority of Conscience

Villette

7. The Isolation of the Foreigner
8. The Concealment of Feeling
9. The Divided Self
10. Faith Under Foreign Custom
11. The Fragility of Fulfillment

The Professor

12. The Ethics of Ambition
13. Intellect Versus Fortune
14. Kinship and Betrayal
15. The Dignity of Labor
16. Rivalry in Courtship

Shirley

17. The Desperation of Poverty
18. Power and Its Limits
19. Loyalty Under Romantic Strain
20. Grief and Public Composure
21. The Uncertainty of Justice

1.2.3 Charlotte Bronte writing style

Charlotte Bronte's writing style usually involves first person narrative. It has long, complex sentences with semi-colons that mimic a flowing stream of consciousness. Combines highly detailed visual descriptions of landscapes and settings with restrained internal dialogue. Think on the page narrative where characters explain the whole thought process. Filters the world through the lens of the narrator. Use latinized english with french expressions here and there. Introspective language where the characters Chain of thought is explained as well.

1.2.4 Text Generation

Gemini paragraph generation

Model used: gemini-3.5-flash-lite

Basic Prompt

“For the topics provided in the {topics } field generate 50 paragraphs. Each paragraph has to be between 100-200 words; these paragraphs can either be of one topic each or be a mix of the topics. The paragraph should be relevant. Remove any starting or ending parts like “Here your response kind of response the response has to be in a json format it should have two parts first part is a list with each entry a textual paragraph like that 50 entries in that list so it should have the key as paragraph, the next entry has to be a maximum 100 word paragraph about the cumulative summaries of the paragraphs generated previously to be passed with the prompt in the next batch to ensure continuity of the content among the paragraphs” so the goal is to generate 500 paragraphs in batches this will be one among the batches the prompt will contain three important inputs the topics for generation, the batch number each batch has 50 entries so first batch number is 1 and last is 10 this information can be used to make sure the paragraphs are relevantly ordered and not randomly generated and finally the prelude batches summary it gets accumulated and will contain the whole context of all the batches as we go down the generation. The paragraphs have to be thematically linked but standalone paragraphs and should not have repetitive paragraphs starters ”

Prompt:

The same prompt was converted into an alpaca format prompt

Instruction:

You are a long-form content generator producing a dataset in 10 sequential batches of 50 paragraphs each (500 paragraphs total). You must strictly follow the response format rules below — violating them breaks an automated parser downstream.

CRITICAL RESPONSE FORMAT RULES (highest priority, must never be violated):

1. Output ONLY a single valid JSON object. Nothing else.
2. Do NOT include any preamble, explanation, acknowledgment, or closing remarks (e.g., no "Here is your JSON", no "I hope this helps").
3. Do NOT wrap the JSON in markdown code fences (no ```json or ```).
4. Do NOT include titles, headers, bullet points, or numbering inside any paragraph text.
5. The output must match this EXACT schema, with these exact keys:

```
{
  "paragraphs": [
    {"paragraph": "<text of paragraph 1>"},
    {"paragraph": "<text of paragraph 2>"},
    ...
    {"paragraph": "<text of paragraph 50>"}
  ],
  "rolling_summary": "<a single paragraph, maximum 200 words>"
}
```

- "paragraphs" must contain EXACTLY 50 objects, each with a single key "paragraph".

- "rolling_summary" must be a single string, max 200 words, that REPLACES all prior context with one fresh, coherent digest covering everything generated so far (all previous batches plus this one). Do not

append to the old summary — rewrite it fresh each time so it stands alone as the complete running context.

CONTENT RULES:

- Topics for this batch: {topics}
- This is batch {batch_number} of 10 total batches (50 paragraphs each, 500 overall). Pace the content as part of a deliberate arc across all 10 batches: early batches (1-2) should establish and introduce ideas, middle batches (3-8) should develop, complicate, and deepen them, and final batches (9-10) should feel like they are moving toward closure and synthesis — without ever explicitly stating meta-commentary like "in conclusion" or "to summarize."
- Each paragraph must be 100-200 words.
- Exactly 60% of the 50 paragraphs (30 paragraphs) must focus on a single topic for each paragraph from the TOPICS list. The remaining 40% (20 paragraphs) must be connecting paragraphs that relate two or more topics together.
- Paragraphs must be standalone (do not use phrases like "as discussed above," "continuing from before," or "as mentioned earlier"), but must be thematically and tonally linked to each other and to the prior context, so the batch reads as a coherent continuation.
- DEDUPLICATION RULE (strict): No two paragraphs, either within this batch or compared against the prior context summary, may start with the same or a near-identical opening sentence structure (e.g., do not repeat patterns like "In today's world..." or "It is important to note..." more than once across the entire batch). Vary sentence openers, paragraph structures, and rhetorical approaches throughout.
- Do not repeat specific phrases, analogies, or examples already used in the prior context summary; introduce new angles, examples, or framing while staying thematically consistent.

BATCH 1 SPECIAL CASE:

If PRIOR_CONTEXT below says "None — this is the opening batch," treat this as the very first batch with no prior history. Do not reference or invent any prior content, and do not generate a summary that implies earlier events occurred.

Input:

TOPICS: {topics}

BATCH NUMBER: {batch_number} of 10

PRIOR_CONTEXT: {prelude_summary}

Response:

Gemini prompt for generating paragraphs mimicking charlotte bronte

PROMPT MODIFICATIONS - with few shot prompting

STYLE RULES (write every paragraph in this voice — highest priority alongside format rules):

- First-person, confessional narration — speak with intimate directness, at times addressing the reader as though speaking to a confidant rather than a distant audience.
- Long, complex sentences built with semi-colons — unspool a single thought across clauses, reaching and curling back on itself, rather than breaking it into short sentences.
- Think-on-the-page narration: narrate the thought process in real time as it unfolds, rather than summarizing conclusions after the fact.

- Filter everything through the narrator's subjective consciousness — events, settings, and other people exist only as the narrator perceives and judges them.
- Combine vivid, highly detailed visual description of landscapes and settings with restrained internal dialogue.
- Realism paired with quiet acceptance — hardship and constraint are faced steadily, never resolved melodramatically.
- Introspective language — frequent self-examination of the narrator's own emotional and moral state.
- Latinate English interspersed with occasional French expressions, lending a formal, educated register.
- Do NOT use modern idioms, casual phrasing, or contemporary vocabulary — maintain a 19th-century literary register throughout.

STYLE EXAMPLES (imitate the SENTENCE CONSTRUCTION, RHYTHM, AND VOICE below — NOT the content, characters, or subject matter; these examples exist purely to demonstrate sentence-level style):

Example 1: "They were not bound to regard with affection a thing that could not sympathize with one amongst them; a heterogeneous thing, opposed to them in temperament, in capacity, in propensities; a useless thing, incapable of serving their interest, or adding to their pleasure; a noxious thing, cherishing the germs of indignation at their treatment, at contempt of their judgement."

Example 2: "I complexly felt that he was of a different nature from me; that his humanitas was not my humanitas; that he was a man of the world, a man of action, a man of code and system, and that I was none of these things. I knew that he looked down on me from a height of intellectual superiority, which he took no pains to conceal; and that he regarded my mind as a piece of raw material, which it was his duty to fashion and polish according to his own notions."

Gemini API response observations

-> The data has an inherent structural pattern. It starts with the theme then an explanation or incident about it. This was noticed in both class 2 and class 3.

-> Class 3 texts follow the style elements of 19th century writing with long sentences split by semicolons.

-> Certain paragraphs are just one long sentence explaining the topic or theme.

-> Repetitive words like labyrinth, human, ethics

Final inference:

The data follows the prompt instructions but exhibits AI generated text characteristics obviously. Given the model capacity and the batch size for generation being 50 I can see a gradual decrease in the quality of paragraph generated towards the end. It can also be coincidental and subjective. The data strictly follows the rules mentioned in the prompt and can be used in the further steps as an initial dataset. The data can be progressively finetuned after initial results.

2. Fingerprinting Data to identify mathematical distinctions between the classes

2.1 Type token ratio:

Find unique tokens/ total tokens for all three classes. The first concern was class 1 consisted of whole novels while the rest had 500 paragraphs each around 60k to 80k tokens. Now as the number of total tokens increases, the possibility of having unique words decreases especially in novels the repetition in the words increases with the size of the novel. So here I have normalised class 1 by mixing core chapters from three books - Jane Eyre, Shirley and Villette. The corpus now has a similar amount of words.

```
# TTR
charlotte_original = get_text_from_files("/content/Charlotte_bronte_original_paragraphs.txt")
ttr = calculate_ttr(charlotte_original)
print(ttr)

charlotte_mimic = get_text_from_files("/content/charlotte_paragraphs.txt")
ttr = calculate_ttr(charlotte_mimic)
print(ttr)

topic_paragraph = get_text_from_files("/content/paragraphs.txt")
ttr = calculate_ttr(topic_paragraph)
print(ttr)

... 0.22335268992657292
0.1281304037631353
0.1412230395212567
```

Inference:

As expected the original human written novel has higher TTR while the AI generated text has lower TTR.

Resources:

<https://medium.com/@rajeswaridepala/empirical-laws-ttr-cc9f826d304d>

2.2 Hapax legomena

The number of words in each class which has the frequency of exactly 1. Here we calculate it in a 5000 word sample of each class.

```
#Hapax legomena
charlotte_original_hl_sample = hapax_legomena(charlotte_original.split()[:5000])
print(charlotte_original_hl_sample)
charlotte_mimic_hl_sample = hapax_legomena(charlotte_mimic.split()[:5000])
print(charlotte_mimic_hl_sample)
topic_paragraph_hl_sample = hapax_legomena(topic_paragraph.split()[:5000])
print(topic_paragraph_hl_sample)

|

... 1565
1469
1413
```

I implemented the hapax legomena function manually. So we pass the word list and calculate the frequency of each word in it and finally pick the words with frequency of one. The count is returned.

In the second level I used the NLTK hapax function along with normalised words. Because in the initially implemented function two same words but one with a punctuation and another without will be considered as different. So we normalise the data to consist of words only with letters and numbers and compare it.

```
from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist

nltk.download('punkt_tab')

def hapax_toolkit(text):

    tokens = word_tokenize(text.lower())
    words = [word for word in tokens if word.isalnum()]
    fdist = FreqDist(words)
    unique_words = fdist.hapaxes()
    return len(unique_words)

... [nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.

(38) charlotte_original_hl_sample = hapax_toolkit(" ".join(charlotte_original.split()[:5000]))
print(charlotte_original_hl_sample)
charlotte_mimic_hl_sample = hapax_toolkit(" ".join(charlotte_mimic.split()[:5000]))
print(charlotte_mimic_hl_sample)
topic_paragraph_hl_sample = hapax_toolkit(" ".join(topic_paragraph.split()[:5000]))
print(topic_paragraph_hl_sample)

... 981
1159
1105
```

```
(35) charlotte_original_hl_sample = hapax_toolkit(" ".join(charlotte_original.split()[:70000]))
print(charlotte_original_hl_sample)
charlotte_mimic_hl_sample = hapax_toolkit(" ".join(charlotte_mimic.split()[:70000]))
print(charlotte_mimic_hl_sample)
topic_paragraph_hl_sample = hapax_toolkit(" ".join(topic_paragraph.split()[:70000]))
print(topic_paragraph_hl_sample)

... 3879
2543
2421
```

Inference:

Here the result honestly surprised me because the original text had fewer hapax words than the AI generated text. I went through the text and realised that the reason behind it is instead because the first 5000 words of the original text being from a book had names, more pronouns and places which kept occurring again and again. Moreover the AI generated texts are instead different paragraphs with less or no continuity between each other. These are paragraphs on a common theme instead of a chapter. So the repetition of words is relatively lesser, especially in such a small sample. I experimented with 100k words instead and the result was as expected with original human text having more unique vocabulary while the AI had lesser.

Resource used:

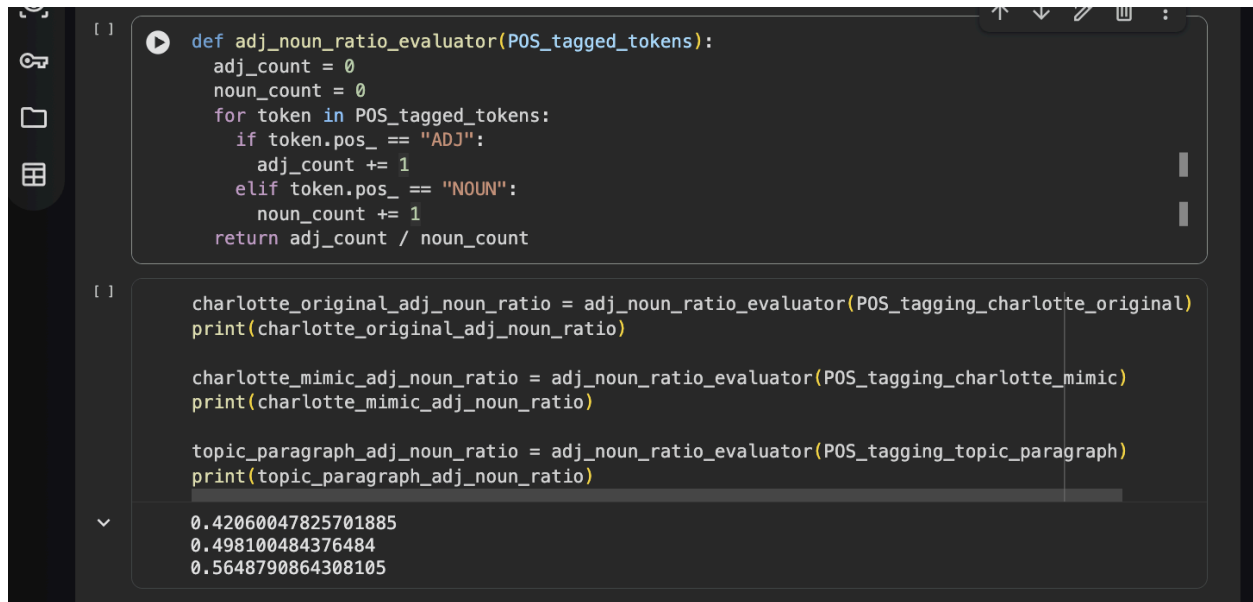
<https://www.textanalyzer.io/blog/hapax-legomena>

<https://www.nltk.org/api/nltk.tokenize.punkt.html>

2.3 Syntactic Complexity (SpaCy/NLTK):

2.3.1 POS Tagging:

Adjective to noun ratio in the classes. Implemented using spacy POS Tagging.



```
[ ] def adj_noun_ratio_evaluator(POS_tagged_tokens):  
    adj_count = 0  
    noun_count = 0  
    for token in POS_tagged_tokens:  
        if token.pos_ == "ADJ":  
            adj_count += 1  
        elif token.pos_ == "NOUN":  
            noun_count += 1  
    return adj_count / noun_count  
  
[ ] charlotte_original_adj_noun_ratio = adj_noun_ratio_evaluator(POS_tagging_charlotte_original)  
    print(charlotte_original_adj_noun_ratio)  
  
    charlotte_mimic_adj_noun_ratio = adj_noun_ratio_evaluator(POS_tagging_charlotte_mimic)  
    print(charlotte_mimic_adj_noun_ratio)  
  
    topic_paragraph_adj_noun_ratio = adj_noun_ratio_evaluator(POS_tagging_topic_paragraph)  
    print(topic_paragraph_adj_noun_ratio)  
  
0.42060047825701885  
0.498100484376484  
0.5648790864308105
```

Inference:

AI tends to overdescribe nouns more than human written text.

Reference:

<https://universaldependencies.org/u/pos/>

<https://spacy.io/api/token#attributes>

<https://www.geeksforgeeks.org/nlp/nlp-part-of-speech-default-tagging/>

2.3.2 Dependency Tree Depth:

Average depth of the dependency tree. It has a root word and all the other words are connected to that word hierarchically at different levels.

```
# depth of a tree calculator

def depth_of_tree(node):
    if len(list(node.children)) == 0:
        return 0
    return 1+max(depth_of_tree(child) for child in node.children)

charlotte_original_depth = [depth_of_tree(sent.root) for sent in POS_tagging_charlotte_origi]
charlotte_original_average_depth = sum(charlotte_original_depth) / len(charlotte_original_de)
print(charlotte_original_average_depth)

charlotte_mimic_depth = [depth_of_tree(sent.root) for sent in POS_tagging_charlotte_mimic.se]
charlotte_mimic_average_depth = sum(charlotte_mimic_depth) / len(charlotte_mimic_depth)
print(charlotte_mimic_average_depth)

topic_paragraph_depth = [depth_of_tree(sent.root) for sent in POS_tagging_topic_paragraph.se]
topic_paragraph_average_depth = sum(topic_paragraph_depth) / len(topic_paragraph_depth)
print(topic_paragraph_average_depth)
```

4.701115485564304
7.735137195121951
7.006888633754305

Inference:

Similarly AI texts are overly complex and hence have greater depth.

Resources:

<https://medium.com/natural-language-processing-nlp/dependency-parsers-in-natural-language-processing-nlp-784654926ccf>

<https://spacy.io/api/dependencyparser>

2.3.5 Readability Indices: Calculate the Flesch-Kincaid Grade Level:

This gives the required US Grade level to read a text. One limitation I noticed is that this text is mostly evaluated using the average words in a sentence and average syllables per word in the text. While that is partially a metric to understand the difficulty of the text there are other factors like the presence of older English vocabulary especially in texts like the 19th century texts.

Flesch Kincaid Grade Level

$$0.39 \left(\frac{\text{total words}}{\text{total sentences}} \right) + 11.8 \left(\frac{\text{total syllables}}{\text{total words}} \right) - 15.59$$

```

total_syllables_human = [word._.syllables_count for word in POS_tagging_charlotte_original if word.is_alpha and word._.syllables_count is not None]
total_words_human = [word for word in POS_tagging_charlotte_original if word.is_alpha]

total_syllables_ai_mimic = [word._.syllables_count for word in POS_tagging_charlotte_mimic if word.is_alpha and word._.syllables_count is not None]
total_words_ai_mimic = [word for word in POS_tagging_charlotte_mimic if word.is_alpha]

total_syllables_topic = [word._.syllables_count for word in POS_tagging_topic_paragraph if word.is_alpha and word._.syllables_count is not None]
total_words_topic = [word for word in POS_tagging_topic_paragraph if word.is_alpha]

Flesch_Kincaid_human = 0.39 * (len(total_words_human) / len(list(POS_tagging_charlotte_original.sents))) + 11.8 * (sum(total_syllables_human) / len(total_words_human)) - 15.59
print("Flesch_Kincaid_human : ",Flesch_Kincaid_human)

Flesch_Kincaid_ai_mimic = 0.39 * (len(total_words_ai_mimic) / len(list(POS_tagging_charlotte_mimic.sents))) + 11.8 * (sum(total_syllables_ai_mimic) / len(total_words_ai_mimic)) - 15.59
print("Flesch_Kincaid_ai_mimic : ",Flesch_Kincaid_ai_mimic)

Flesch_Kincaid_topic = 0.39 * (len(total_words_topic) / len(list(POS_tagging_topic_paragraph.sents))) + 11.8 * (sum(total_syllables_topic) / len(total_words_topic)) - 15.59
print("Flesch_Kincaid_topic : ",Flesch_Kincaid_topic)

# Not a right metric honestly because doesnt track more essential metrics like the langugae just the syllables cant be a marker right

Flesch_Kincaid_human : 7.9855133542595205
Flesch_Kincaid_ai_mimic : 15.734811610038069
Flesch_Kincaid_topic : 15.10447532071385

```

Inference:

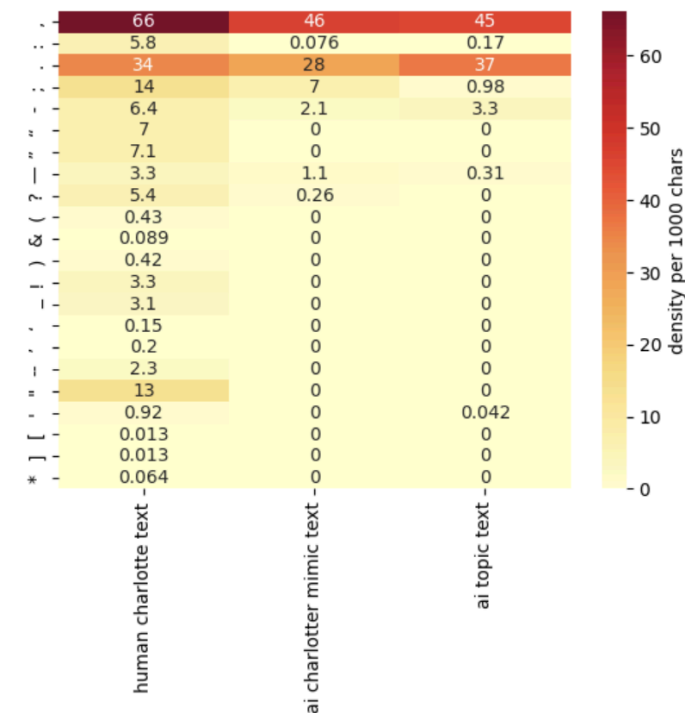
Here the human text is at grade level of 8 while AI text generated on topics discussed in the original text offered complicated academic paper level grading of 15. This is an expected scoring since the AI is trained on a humongous amount of academic and professional papers than usual layman data.

Resources:

https://en.wikipedia.org/wiki/Flesch%E2%80%93Kincaid_readability_tests

https://spacy.io/universe/project/spacy_syllables

2.3.3 Punctuation Density Heatmap



Inference:

As expected Human text has more punctuations in general than AI generated text. AI generated text mostly has similar values except for semi-colons in AI Mimic charlotte bronte. We specifically prompted the model to generate long sentences with semicolons so it has comparatively more semicolons than generally generated AI text. And AI has a more limited punctuation list than Human AI text.

Table:

Metric	Class 1	Class 2	Class 3
Type Token Ratio	0.2234	0.1281	0.1412
Hapax Legomena	981	1159	1105
POS Distribution	0.420	0.499	0.564
Dependency Tree Depth	4.7011	7.7351	7.006
Punctuation Density			
Readability Indices	7.9855	15.7348	15.1044

3. AI Detection models

3.1: Data Preparation:

Paragraph is the unit of evaluation in this experiment. And hence to make sure the model doesn't learn to classify based on random parameters like length, we ensure that the paragraphs are of uniform length.

Train - Test Split - standard(80|20) - 500 paragraphs - 400 Train / 100 Test

Human Text:

-> Pass all the paragraphs in the books and make a list of the paragraphs that have word length between 80 - 230 words. Of these, based on the book size I have chosen representation from each book to make sure it covers all the topics evenly.

160 paragraphs from Jane Eyre picked from start, end and middle

160 paragraphs from Vilete picked from start, end and middle

80 paragraphs from Professor picked from start and end

100 paragraphs from Shirley picked from start and end

In this Jane Eyre, Vilete and Professor excerpts are used as training while Shirley is picked as a testing set.

AI Texts: Class 2 & 3 we perform a split in 500 paragraphs and make it 400 train and 100 test data.

3.2 Random Forest classifier:

-> Random forest has n- number of decision trees. It is built using mathematical features based on which the model learns on the behaviour of different classes on the feature parameters

Here the parameters are the fingerprints we evaluated earlier

- TTR (Type token Ratio)
- Hapax legomena rate
- POS Distribution
- Average Dependency tree depth
- Punctuation density
 - Em-dashes Density
 - Exclamation density
 - Semi-colon Density
 - Other punctuations density
- Readability indices

-> First we create a feature file for the training and testing set. The training set has all the training split data along with class id followed by the above values for each paragraph.

Class 1 = Charlotte bronte excerpts
 Class 2 = AI mimic bronte excerpts
 Class 3 = AI excerpts

-> Using the above data we train a random forest model with 100 trees.

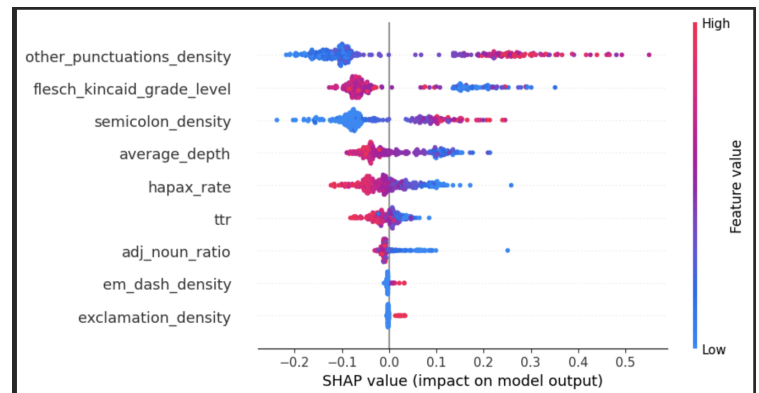
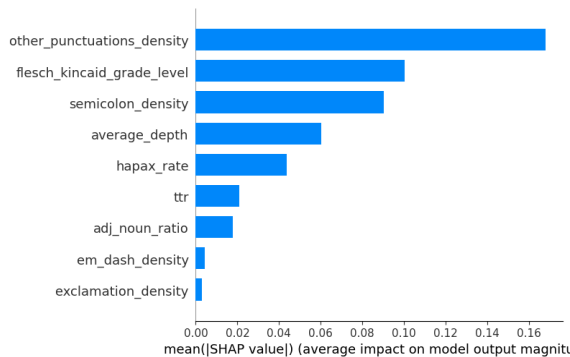
The number of trees was chosen after a set of trial and error. The current value was an optimal choice with accuracy reaching optimal value of 85.33%

Confusion Matrix

[[97 2 1]					
[2 68 30]					
[1 8 91]]					
	precision	recall	f1-score	support	
1	0.97	0.97	0.97	100	
2	0.87	0.68	0.76	100	
3	0.75	0.91	0.82	100	
accuracy			0.85	300	
macro avg	0.86	0.85	0.85	300	
weighted avg	0.86	0.85	0.85	300	

SHAP Analysis on the model:

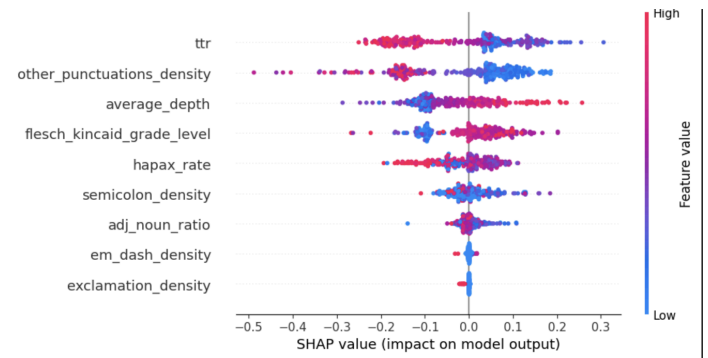
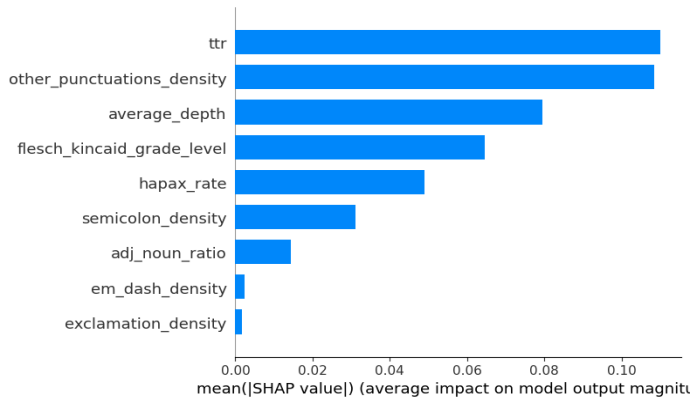
Class 1 - Human Text



Inference:

The human text had relatively more types of punctuations than AI generated texts which were mostly full stop, comma, semi colon, em-dashes and exclamations. So naturally that is a significant factor in the model output. And the readability score is relatively lesser for human text than AI models because AI words have more syllables than common human words.

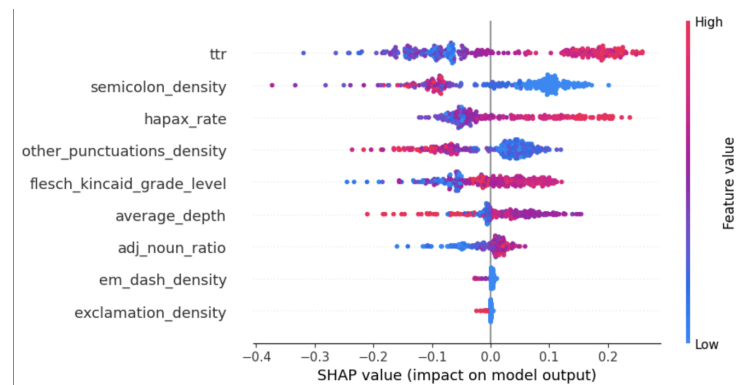
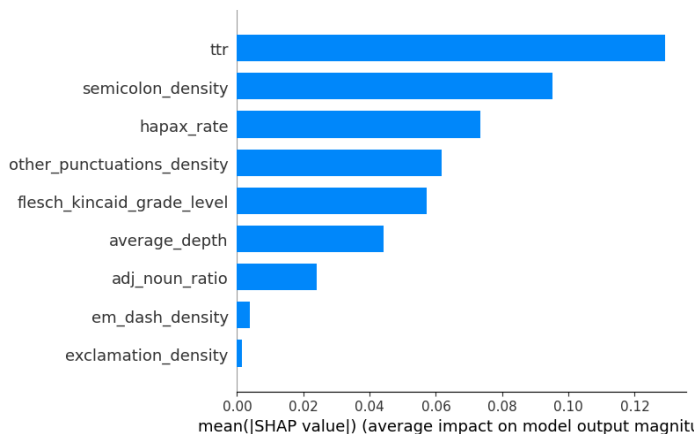
Class 2: AI mimicking human



Inference:

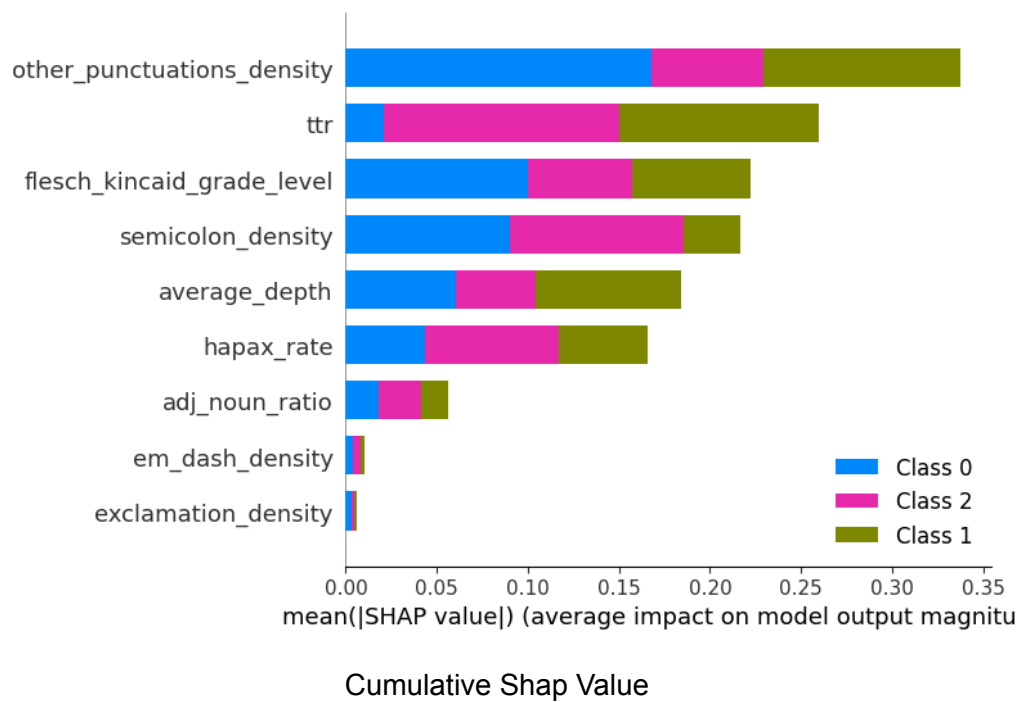
The AI text mimicking humans has relatively lower TTR value and has lesser punctuation density. This is supported along with forced long sentences with semicolons making the average depth higher.

Class 3: AI Text



Inference:

AI text has more unique words in a single paragraph along with very less semicolon density. This can be a major difference between AI generated and AI mimicking human language. Because in the AI mimicking we voluntarily enforce it to use more semi-colons.

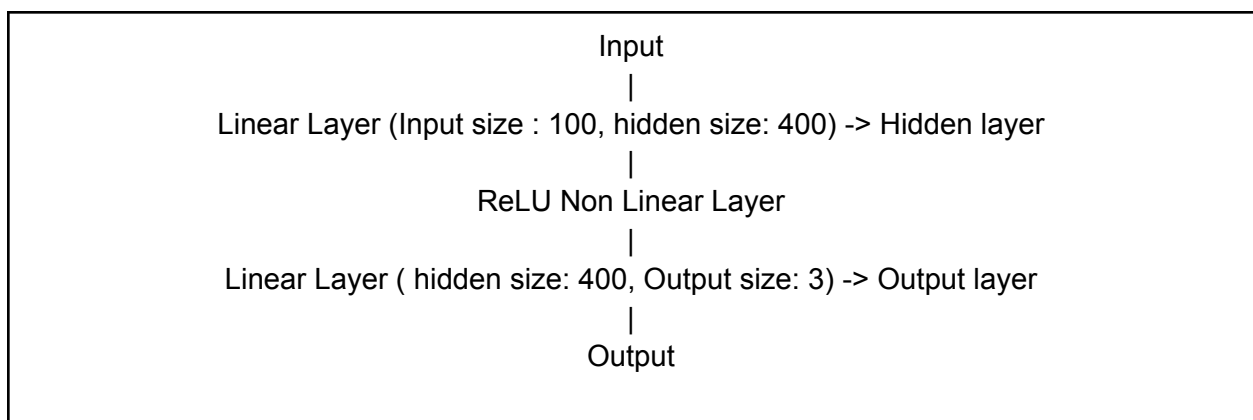


2.3 FeedForward Neural Network

Architecture:

Embedding = FastText embedder

Simple MLP with a single hidden layer.



Steps:

- > Tokenise all the train paragraphs
- > Initialize fastText model with the Tokenised paragraph
- > Create embedding for each paragraph by averaging the vector for each token in the paragraph. Each paragraph is a 100 dimension vector. (100 is the standard dimension size for larger data corpus we can increase the dimension size to capture more features)
- > Initialize X_Train = embeddings for paragraphs Y_Train = labels of the paragraphs similarly X_Test and Y_Test
- > Initialize the feedforward neural network
- > Train the model for 50 epoch (Hyperparameter not tuned just a random initial value)

Results

[[99 1 0] [0 91 9] [0 3 97]]					
	precision	recall	f1-score	support	
Class 1	1.00	0.99	0.99	100	
Class 2	0.96	0.91	0.93	100	
Class 3	0.92	0.97	0.94	100	
accuracy			0.96	300	
macro avg	0.96	0.96	0.96	300	
weighted avg	0.96	0.96	0.96	300	

Resources:

<https://fasttext.cc/docs/en/python-module.html>

<https://www.geeksforgeeks.org/machine-learning/fasttext-working-and-implementation/>

<https://www.datacamp.com/tutorial/feed-forward-neural-networks-explained>

<https://ai.plainenglish.io/ffnn-lm-feed-forward-neural-network-language-model-8925f7e2a0ab>

https://www.deeplearningwizard.com/deep_learning/practical_pytorch/pytorch_feedforward_neuralnetwork/#parameters-in-depth

<https://medium.com/@abhishekjainindore24/fasttext-embedding-technique-9b5d1fba4b50>

RoBERTa Finetuned with LoRA

RoBERTa (Robustly optimised BERT Pretraining Approach) is built on BERT(encoder only model), modifying certain hyper parameters and removing next sentence prediction and introducing dynamic masking on it instead of static masking.

For this classification task we use the RoBERTaForSequentialClassification model which is an additional classification head over the RoBERTa model to classify the text into classes by providing probability scores for the text belonging to each class. On this classification model we fine-tune with LoRA using our data to enable it to perform Machine Generated Text Detection.

The first step in this process is to convert the paragraphs into roberta format tokens - with <s> special tokens, Byte level tokenisation, padding and truncation. We use RobertaTokenizer for this process. RobertaTokenizer has two variants: the Python base variant and fast variant with Rust with python binding. In my cached downloads the Fast variant did not work as expected so I had to stick to the base variant. The only major trade off is the speed but that matters in scale here for 1500 paragraphs with 200 words max doesn't make much difference. Next we encode the train and test dataset with a tokenizer. The fine tuning works better with a validation set to evaluate between epochs so the training set was further split into 320 training and 80 validation per class.

After the set up for RoBERTa we set up LoRA for fine tuning (Low Rank Adaptation) so the core idea of this algorithm is that the features represented by a model can be represented with a lower intrinsic dimension. So that means to finetune a model to perform a certain task we don't have to retrain the whole model rather we freeze the pretrained model and train a smaller number of parameters which will later be added to the frozen model through linear operations.

The LoRA is set up with $r=8$ and $\alpha = 32$; these are optimal values. But can be tuned later based on performance. The value r is the rank of the parameter matrix while α is the volume of impact. After attaching lora to the model the trainable parameters went down to 0.82 % from 100 %. This represents the smallest fraction we require to train this model.

Post training this model gave an accuracy of 98.33%.

```
train_labels = torch.tensor(train_labels)
TEST RESULTS
=====
Accuracy: 0.9833
Precision: 0.9841
Recall: 0.9833
F1: 0.9833

True label distribution:
[100 100 100]

Predicted label distribution:
[100 95 105]

Confusion Matrix:
[[100  0  0]
 [  0 95  5]
 [  0  0 100]]
```

During this training, the first few times the model completely collapsed and was constantly classifying all the input as class 3. After performing various diagnostics on the input, base model and the loRA attached model. I finally found that the learning rate was the problem. Based on my previous feed forward network training I set the learning rate as 0.005 but this led to catastrophic forgetting of the generalised learnings in the pre-trained model. So the ideal rate for fine tuning a model ranges around $2e-5$.

Resources:

https://huggingface.co/docs/transformers/v4.48.0/en/model_doc/roberta#transformers.RobertaForSequenceClassification

https://huggingface.co/docs/transformers/main/tasks/sequence_classification?utm_source=chatgpt.com

<https://arxiv.org/abs/1907.11692>

https://medium.com/@marketing_novita.ai/introducing-roberta-base-model-a-comprehensive-overview-330338afa082

<https://medium.com/analytics-vidhya/create-a-tokenizer-and-train-a-huggingface-roberta-model-from-scratch-f3ed1138180c>

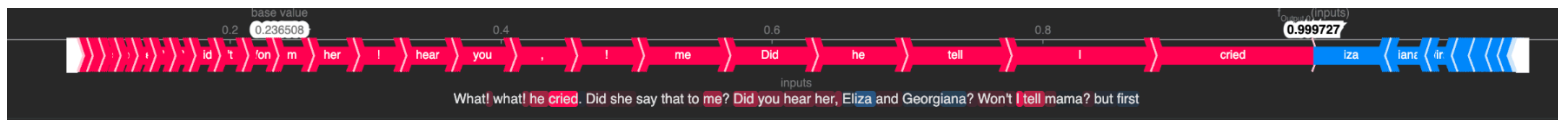
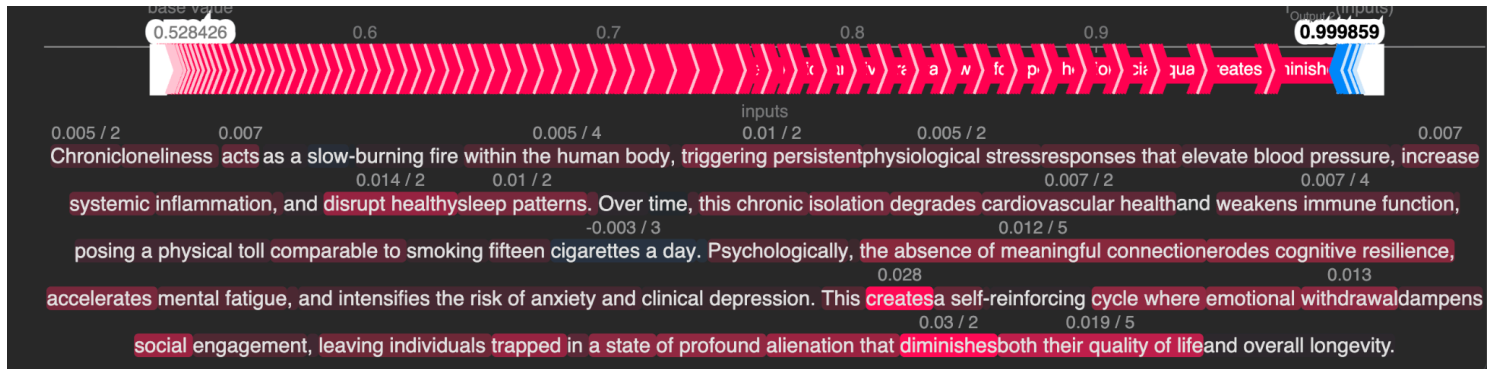
<https://huggingface.co/docs/peft/en/install>

<https://www.kaggle.com/code/anthonymam/lora-fine-tuning-with-roberta-daigt-v2>

<https://huggingface.co/learn/llm-course/en/chapter3/3>

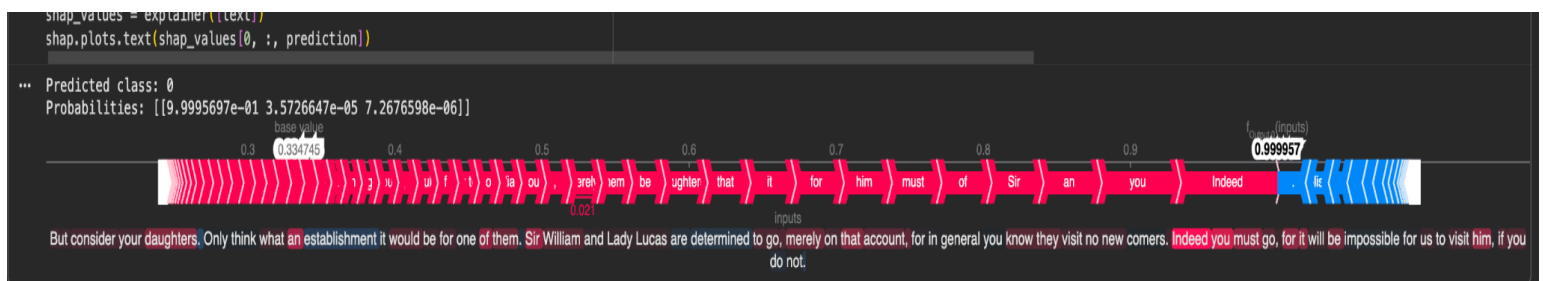
4 .SHAP Analysis on this model:

Correct Predictions:



This is a paragraph from another 19th century book and this got classified correctly.

But consider your daughters. Only think what an establishment it would be for one of them. Sir William and Lady Lucas are determined to go, merely on that account, for in general you know they visit no newcomers. Indeed you must go, for it will be impossible for us to visit him, if you do not.

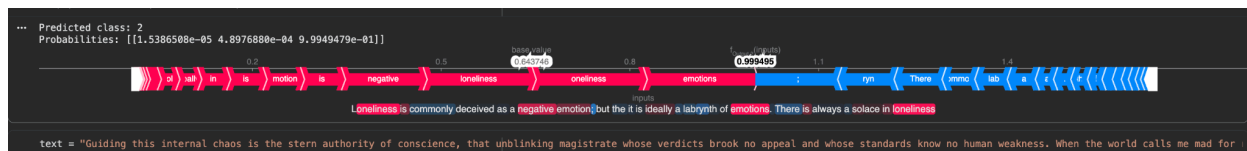


Inference:

The model seems to consider various parameters, not just words. This paragraph unanimously votes towards class 0. Especially its driven by Nouns and pronouns. The highest contributing word is Indeed this can either point out that the model looks into the position of the word doesn't match the usual AI usage of the word or there might be some other syntactical reason.

Imposter paragraphs

I wrote the below entry about loneliness and the model classified it as AI. I infer that this is majorly about the dataset because since the topics directly included loneliness and isolations the generated paragraphs had more frequency of these words driving the model to learn that.

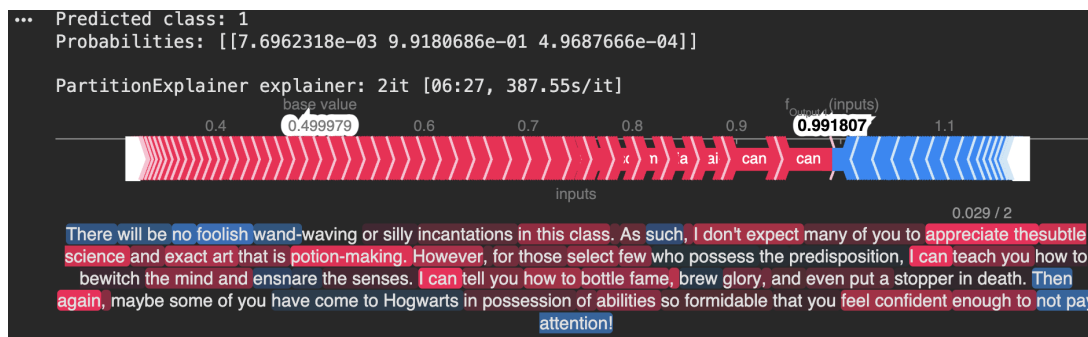


To experiment more with the texts I tried a current generation short story excerpt. And it classified it as class 1. Now the text follows the first person narrative and the 21st century language which is more similar to class 1.

The dahlias are in the fridge. He must have bought them at the supermarket that day. A split-second decision or a planned and sweet intention. I don't know. And then he must have packed them away with the food. With the pale turkey and winter vegetables scrubbed clean of earth. The flowers are a stark slash of colour. Chilled and forgotten.



Similarly harry potter excerpts got classified as class 1 with phrases like potion-making, subtle science voting for the class. And once again the first person narrative drives the model prediction.



This was an interesting find where this paragraph is excerpted from a well written research paper. Ideally because of the names of the phrases like the reader. I expected the model to wrongly classify it as class 1 but the model has classified it as class 2 (General AI text) which is a win because the model is biased on the language style but has captured the underneath pattern in the



Error analysis:

The error in the model is mostly attributed towards the diversification of the data. Since the human text data was a 19th century text. The model rather learnt the written styles with that language. The current more evolved linguistics are classified as AI since it was more relatable to the language in AI generated datasets. One interesting finding is that the model cleanly classifies excerpts from recent books as AI mimicking humans instead of just AI. That can be used to infer that the model has learnt the narrative styles and patterns.

5. Genetic Algorithm:

For the super imposter task, the initial population I chose was the class 2 entries. I generated a fresh batch of class 2 texts which are AI generated but closely mimicking charlotte bronte. The initial population had 10 AI generated paragraphs. The fitness scores are human class probabilities for each text. Of these we select the top 3.

The first selection had these fitness scores : max score: 0.0053

```
print(fitnesses)
print(len(selected))

... [np.float32(0.005388513), np.float32(0.00081565726), np.float32(0.0005548922),
3
```

After this we perform the mutation with LLM as the mutator. The mutator function has multiple strategies

```
MUTATION_STRATEGIES = [
    "Rewrite to change sentence rhythm and length while keeping vocabulary the same.",
    "Introduce one subtle grammatical inconsistency or one rare archaic word.",
    "Vary punctuation usage - add or remove semicolons, dashes, or comma placement.",
    "Rewrite using more Latinate vocabulary and formal register.",
]
```

Here during each iteration it chooses one strategy and mutates the text. The prompt used for this is

```
### Instruction
You are a 19th century English author. You use {current_strategy} to
rewrite the paragraph you receive. Perform the task below and give the response
strictly following the response format.

### RESPONSE CONTENT RULES(very important)

1) Retain the meaning of the paragraph.
2) Each paragraph has to be rewritten and given a individual object
3) DON'T LEAVE OUT ANY PARAGRAPH

### CRITICAL RESPONSE FORMAT RULES (highest priority, must never be violated):
1. Output ONLY a single valid JSON object. Nothing else.
2. Do NOT include any preamble, explanation, acknowledgment, or closing remarks.
```

3. Do NOT wrap the JSON in markdown code fences.
4. Do NOT include titles, headers, bullet points, or numbering inside any paragraph text.
5. The output must match this EXACT schema, with these exact keys:

```
{
  "paragraphs": [
    {"paragraph": "<rewritten text of paragraph 1>"},
    {"paragraph": "<rewritten text of paragraph 2>"},
    {"paragraph": "<rewritten text of paragraph 3>"}
  ]
}
```

INPUT

PARAGRAPHS: {population}
STRATEGY: {current_strategy}

RESPONSE

The mutation gave really good improvement in the probability values after the first iteration.

```
print(fitnesses)

[np.float32(0.011638985), np.float32(0.005388513), np.float32(0.0011158049)]
```

After 10 iterations the fitness score improved to

```
fitness_top = [fitness_function(text) for text in selected_top_3]
print(fitness_top)

... [np.float32(0.1396752), np.float32(0.025995784), np.float32(0.016535211)]
```

The score improved from 0.005 to 0.140. This improvement is really affirmative in the direction of our research. The change from baseline value to the current stage is a huge jump and is promising.

Resources:

<https://www.datacamp.com/tutorial/genetic-algorithm-python>

Future work:

The future work on this project is essentially fine-tuning the dataset first. Right now the model believes that the human writing style is 19th century victorian writing style we can change that by providing more human written texts in current linguistic styles. Similarly better identification of human styles can help in prompting the AI model to generate texts that have higher probability values of getting classified as human writing,

Experience through this experiment

I started this experiment with a clean slate. This was my first time working on an end to end NLP Task. Each step had me looking at a lot of resources. There were many steps where the code broke or the methodology failed. The experience of debugging and fixing the errors and understanding the libraries used was truly enlightening. I would love to receive feedback on the project and keep improving

Thanks for the opportunity - Regards Srinidhi